If you get tired of typing bundle exec and you are using gems that don't autoload when you run them, like, say RSpec, Bundler offers *binstubs*, which are a way for Bundler to create a wrapper program around a specific executable Ruby program defined by your gems that invokes Bundler before it starts.

As you'll see in Writing and Packaging Your Own Code Into Gems, on page 261, gems can signal to their users that there are top-level executable commands that are part of the gem. RSpec, the gem, offers rspec the command as its test runner. But if we don't want to keep typing bundle exec rspec, we can ask Bundler to create a binstub for us.

```
$ bundle binstubs rspec
rspec has no executables, but you may want one from a gem it depends on.
  rspec-core has: rspec
```

Oops. Turns out the rspec gem itself has almost no code, it's just a way to get all the dependent pieces of RSpec in one place. Let's try again:

```
$ bundle binstubs rspec-core
```

There's no output here, but we now have a file at bin/rspec. (If you call bundle binstubs and don't specify a gem, it'll run through all the gems in your bundle and create all their binstubs.)

The generated binary file has some boilerplate to make sure the environment is correct, but the main part is at the end:

```
require "rubygems"
require "bundler/setup"

load Gem.bin_path("rspec-core", "rspec")
```

It requires bundler/setup and then loads the rspec script via RubyGems. As a result, you now have a bin/rspec that behaves the same as bundle exec rspec.

If you don't think that's much better, what you can then do is set up your Unix shell's $PATH variable with something like export PATH="./bin:$PATH", and then you can just use $ rspec like before and the version in ./bin will be found first and executed. If this paragraph doesn't make sense, check out *Appendix 3, Command-Line Basics*, for context.

Be default, when you engage Bundler via Bundler.require or require bundler/setup each gem will be autoloaded under the name of the gem. In other words, gem rspec implies require rspec. There are cases where this behavior is not helpful. Sometimes the gem actually has a require file that is not the same as the name of the gem (this was much more common in the past, but gems have tended to converge on the convention that makes using Bundler easier). Sometimes the gem has undesirable behavior when loaded at the top of the program and you only want to require it at a specific point—perhaps it does extra logging or something.

You can control the require behavior by augmenting the line of gem code with a require: keyword option. If the option is false, then the gem is not auto required:

```
gem "rspec", require: false
```

If the option is a string, then the gem is auto-required, but using the name of the require option rather than the name of the gem. More rarely, if the gem is an array of strings, each of those names are required when Bundler autoloads.